Type Theory in a Lean Context

An Alternative Foundation to Mathematics

Naveen Maurya, IISc

October 12, 2026

Feynman Lecture Hall, ICTS

Table of contents

1. Foudational Systems

2. (Martin-Löf) Type Theory

3. Logic in Type Theory

4. Mathematics in Type Theory

5. Conclusion

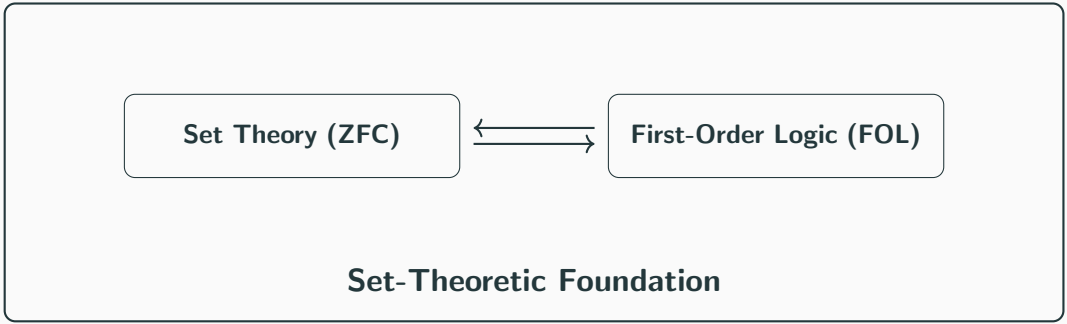
Foudational Systems

What is a Foundational System?

Set Theory is not an 'optimal' foundation for mathematics.

- Set theory depends on first-order logic to state it, and first-order logic usually requires set theory to build.
- Set theory has a problem about interpreting *types* of mathematical objects.
  For example, $\{\{\}, \{\{\}\}\}$ could mean either 2 or the ordered pair $(0, 0)$.
- Set theory is uniform rather than structural.
- Many abuses of language, making it hard to use on computers.

What is a Foundational System?



What is a Foundational System?

Concretely, a Foundation System consists of

- A **Logical/Deductive system** specifying the *langauage of the system* and what counts as *valid reasoning*.
- An **Axiomatic system**, a schematic collection of *axioms*, statements in the language of the deductive system, taken as true by assumption.

Type Theory is both of these together!

(Martin-Löf) Type Theory

Types, Terms and Universes

Types and *Terms* are primitive objects.

- Every term has a unique type.
- Say a has type A. We also a is an *instance* of A, and A is inhabited by a.
- Certain terms may be *judgementally equal* to other terms of the same type, denoted $a \equiv b$.
- Underlying *inference rules* allow transitioning from a context to another by making new *judgements*.
- Judgements are of two forms:
  - $a : A$, called *typing judgements*,
  - $a \equiv b : A$, called *equality judgements*.
- Every type A is a term of some *universe* $\mathcal{U}$.

Universes

Type Theory postulates a chain of cumulative universes in the style of Russell:

$$\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \mathcal{U}_3, \ldots$$

satisfying:

- $\mathcal{U}_i : \mathcal{U}_{i+1}$ for each i
- If $A : \mathcal{U}_i$, then $A : \mathcal{U}_{i+1}$

Each type is a term of some universe. Each universe is also a term of universes above it.

Logic in Type Theory

Embedding a Logic through the Construction Material

Type Theory repurposes the available bricks for building math to also encode logic!

- Certain constructions and manipulations (type formers) of *bricks* are available in the system.

Certain types, called *mere propositions* may be used to capture logical propositions.
One may intepret terms of these types as *proofs* to the proposition.
Hence, proofs of statements are themselves first-class mathematical objects.

Type Formers

Type Former	Assumes	Notation	Logical Analogue
Function Type	$A, B : \mathcal{U}$	$A \to B$	$A \implies B$
Dependent Function Type	$A : \mathcal{U}, B : A \to \mathcal{U}$	$\prod_{(x:A)} B(x)$	$\forall x \in A, B(x)$
Dependent Pair Type	$A : \mathcal{U}, B : A \to \mathcal{U}$	$\sum_{(x:A)} B(x)$	$\exists x \in A, B(x)$
Product Type	$A, B : \mathcal{U}$	$A \times B$	$A \wedge B$
Coproduct Type	$A, B : \mathcal{U}$	$A + B$	$A \vee B$
Empty Type		$\mathbf{0}$	`False`
Unit Type		$\mathbf{1}$	`True`

Constructivity of Type Theory

Type Theory is a constructive foundational system!

- A function $f : A \to B$ needs to have an alogorithm to effectively output an element $f(a) : B$ when given input $a : A$.
- $p \vee q$ requires either a proof of p or a proof of q.
- A proposition $\exists a : A, B(a)$ needs a an explicit $a : A$ such that $B(a)$ holds as a certificate.

Curry-Howard Correspondence b/w proofs and programs!
Type Theory requires proofs to be *computationally meaningful*.

A Note on "Currying"

Function of multiple variables are often written as function into function types as follows:

$$\mathrm{Hom}(A \times B, C) \cong \mathrm{Hom}(A, B \to C)$$
$$f \mapsto \widetilde{f}$$
$$f(a, b) \mapsto f(a)(b)$$

And convention write without brackets if on the right:

$$A \to B \to C \equiv A \to (B \to C)$$

but not $(A \to B) \to C$.

Mathematics in Type Theory

Inductive Types

Natural numbers are the *model mathematical object* in Type Theory!

Remember Peano Axioms for $\mathbb{N}$,

- There is a distinguished element $0 \in \mathbb{N}$.
- There is an injective function $\mathtt{succ} : \mathbb{N} \to \mathbb{N}$,
- There is no $n \in \mathbb{N}$ such that $\mathtt{succ}(n) = 0$.
- The *induction principle*, i.e. to show a predicate P on $\mathbb{N}$ holds for all $n \in \mathbb{N}$, it suffices to prove $P(0)$ holds, and that for all $n \in \mathbb{N}, P(n) \implies P(\mathtt{succ}(n))$.

There's a somewhat obvious-looking property one can prove:

Theorem (Classification of Natural Numbers)
Any $n \in \mathbb{N}$ is either 0 or $\mathrm{succ}^m(0)$ for some $m \in \mathbb{N}$.

Proof.
Let $P(n)$ be the statement "$n = 0$ or there exists some $m \in \mathbb{N}$ such that $\mathrm{succ}^m(0) = n$."
By the induction principle, it suffices to show $P(0)$ and $\forall n \in \mathbb{N}, P(n) \implies P(\mathrm{succ}(n))$.

$P(0)$ is clearly true, and for some $n \in \mathbb{N}$, suppose $n = \mathrm{succ}^m(0)$, then note
$\mathrm{succ}(n) = \mathrm{succ}(\mathrm{succ}(m)(0)) = \mathrm{succ}^{\mathrm{succ}(m)}(0)$. □

Inductive Types

There is a general schema for constructing new types, requiring:

- **Formation rules** to introduce an inductive type into context.
- **Constructors/introduction rules** to introudce terms of the type.
- **Elimination rules** to consume terms of a type.
  (Usually by an *induction* principle which defines dependent functions on the type)
- **Computation rules** to specify action of elimination on the constructors.

These rules specify the behaviour of an inductive type by dictating the inference rules for the type in the foundational system.

Examples

Natural Numbers Type ($\mathbb{N}$) The natural numbers are an inductive type, given by the constructors

- $\mathtt{zero} : \mathbb{N}$
- $\mathtt{succ} : \mathbb{N} \to \mathbb{N}$

and the induction principle that for $E : \mathbb{N} \to \mathcal{U}$, to prove $\prod_{(n:\mathbb{N})} E(n)$ is equivalent to proving $E(0)$ and $\prod_{(n:\mathbb{N})} E(n) \to E(\mathtt{succ}(n))$. Formally,

$$\mathbf{ind}_{\mathbb{N}} : \prod_{(E:\mathbb{N}\to\mathcal{U})} \left(E(0) \times \prod_{(n:\mathbb{N})} (E(n) \to E(\mathtt{succ}(n))) \right) \to \prod_{(n:\mathbb{N})} E(n)$$

Examples

The Empty Type (0) The empty type $\mathbf{0}$ is given by no constructors. One has the universal function for any $E : \mathbf{0} \to \mathcal{U}$, one has the universal function $\prod_{(z:\mathbf{0})} E(z)$. Formally,

$$\mathbf{ind}_0 : \prod_{(E:\mathbf{0}\to\mathcal{U})} \prod_{(z:\mathbf{0})} E(z)$$

Examples

The Unit Type (1) The unit type $\mathbf{1}$ is given by one constructor,

- $* : \mathbf{1}$.

and the induction principle that for $E : \mathbf{1} \to \mathcal{U}$, to prove $\prod_{(z:\mathbf{1})} E(z)$, it suffices to prove $E(*)$. Formally,

$$\mathbf{ind}_1 : \prod_{(E:\mathbf{1}\to\mathcal{U})} E(*) \to \prod_{(z:\mathbf{1})} E(z)$$

Examples

The Coproduct Type ($A + B$) The coproduct type of A and B is an inductive type, given by the constructors

- $\mathtt{inl} : A \to A + B$
- $\mathtt{inr} : B \to A + B$

and the induction principle that for $E : A + B \to \mathcal{U}$, to prove $\prod_{(z:A+B)} E(z)$ is equivalent to proving $E(\mathtt{inl}(a))$ for all $a : A$ and $E(\mathtt{inr}(b))$ for all $b : B$. Formally,

$$\mathbf{ind}_{A+B} : \prod_{(E:A+B\to\mathcal{U})} \left(\prod_{(a:A)} E(\mathtt{inl}(a)) \times \prod_{(a:A)} E(\mathtt{inl}(a)) \to \prod_{(z:A+B)} E(z) \right)$$

A Custom Inductive Type

Binary Trees Given a type $\mathtt{A} : \mathcal{U}$, the type of binary trees over $\mathtt{A}$ is $\mathtt{BinTree}(\mathtt{A})$ given by the constructors

- $\mathtt{leaf} : \mathtt{A} \to \mathtt{BinTree}(\mathtt{A})$
- $\mathtt{node} : \mathtt{BinTree}(\mathtt{A}) \to \mathtt{BinTree}(\mathtt{A}) \to \mathtt{BinTree}(\mathtt{A})$

with the induction principle, that for $E : A \to \mathcal{U}$, to prove $\prod_{(z:\mathtt{BinTree}(\mathtt{A}))} E(z)$, it suffices to prove $\prod_{(a:A)} E(\mathtt{leaf}(\mathtt{a}))$ and $\prod_{(t_1,t_2:\mathtt{BinTree}(\mathtt{A}))} E(\mathtt{node}(t_1)(t_2))$.

Structural Constructions

Magma A *magma* is a type M with a binary operation $\cdot : M \to M \to M$. The type *Magma* is given by

$$\mathit{Magma} :\equiv \sum_{(M:\mathcal{U})} (M \to M \to M)$$

Semigroup A *semigroup* is a magma with the operation being associative

$$\mathit{Semigroup} :\equiv \sum_{(M:\mathcal{U})} \sum_{(\cdot : M \to M \to M)} \prod_{(a,b,c:M)} (a \cdot b) \cdot c = a \cdot (b \cdot c)$$

Equality

What does it mean for two mathematical objects to be equal?
In type theory, there are two types of equalities (say between terms a and b):

- **Judgemental Equality** ($a \equiv b$) Equality of terms within the foundational system.
- **Propositional Equality** ($a = b$) Equality on the level of mathematical objects.

Identity Types

Propositional equality in Type Theory is handled by Inductive Types as well! Suppose $A : \mathcal{U}$.

- Then for any $x, y : A$, we have a type $\mathrm{Id}_A(x, y) \equiv (x =_A y) : \mathcal{U}$.
- For any $a : A$, there is the reflexive $\mathrm{refl}_a : a =_A a$.
- There is a *path induction* principle.

Note since $\mathrm{Id}_A(x, y)$ is a type, we can have an identity type on it, $\mathrm{Id}_{\mathrm{Id}_A(x,y)}(p, q)$ for $p, q : x =_A y$, then $\mathrm{Id}_{\mathrm{Id}_{\mathrm{Id}_A(x,y)}(p,q)}(h_1, h_2) \ldots$ ad infinitum.

Conclusion

Summary

- Type Theory is a versatile, constructive foundational system.
- Logic in Type Theory is obtained by interpreting types as propositions and their terms as proofs.
- Inductive Types allow one to build mathematics in Type Theory with native support for structural architecture and computational convenience.
- An Inductive Type is described by a list of constructors, which *freely generate* the type, and are accompanied by an *induction principle*.
- Equality of mathematical objects is described by *Identity Types*.

References & Further Reading

[1] Martin Hofmann. **“Syntax and semantics of dependent types”.** In: *Extensional Constructs in Intensional Type Theory*. Springer, 1997, pp. 13–54.

[2] Per Martin-Löf. **An intuitionistic theory of types: Predicative part.** Vol. 80. Elsevier, 1975, pp. 73–118.

[3] The Univalent Foundations Program. **Homotopy type theory: Univalent foundations of mathematics.** The Univalent Foundations Program, 2013.

Thank You!